Inferencing with Bayesian Network in Python

In this demonstration, we'll use Bayesian Networks to solve the well-known Monty Hall Problem. Let me explain the Monty Hall problem to those of you who are unfamiliar with it:

This problem entails a competition in which a contestant must choose one of three doors, one of which conceals a price. The show's host (Monty) unlocks an empty door and asks the contestant if he wants to swap to the other door after the contestant has chosen one.

The decision is whether to keep the current door or replace it with a new one. It is preferable to enter by the other door because the price is more likely to be higher. To come out from this ambiguity let's model this with a Bayesian network.

Implementation

For this demonstration, we are using a python-based package pgmpy is a Bayesian Networks implementation written entirely in Python with a focus on modularity and flexibility. Structure Learning, Parameter Estimation, Approximate (Sampling-Based) and Exact inference, and Causal Inference are all available as implementations.

```
1 !pip install pgmpy
```

```
Looking in indexes: https://pypi.org/simple, https://us-python.pkg.dev/colab-wheels/public/simple/
Collecting pgmpy
  Downloading pgmpy-0.1.19-py3-none-any.whl (1.9 MB)
     |████████████████████████████████| 1.9 MB 4.6 MB/s
Requirement already satisfied: joblib in /usr/local/lib/python3.7/dist-packages (from pgmpy) (1.1.0)
Requirement already satisfied: scipy in /usr/local/lib/python3.7/dist-packages (from pgmpy) (1.7.3)
Requirement already satisfied: pyparsing in /usr/local/lib/python3.7/dist-packages (from pgmpy) (3.0.9)
Requirement already satisfied: tqdm in /usr/local/lib/python3.7/dist-packages (from pgmpy) (4.64.0)
Requirement already satisfied: numpy in /usr/local/lib/python3.7/dist-packages (from pgmpy) (1.21.6)
Requirement already satisfied: scikit-learn in /usr/local/lib/python3.7/dist-packages (from pgmpy) (1.0.2)
Requirement already satisfied: torch in /usr/local/lib/python3.7/dist-packages (from pgmpy) (1.12.0+cu113)
Requirement already satisfied: pandas in /usr/local/lib/python3.7/dist-packages (from pgmpy) (1.3.5)
Requirement already satisfied: statsmodels in /usr/local/lib/python3.7/dist-packages (from pgmpy) (0.10.2)
Requirement already satisfied: networkx in /usr/local/lib/python3.7/dist-packages (from pgmpy) (2.6.3)
Requirement already satisfied: python-dateutil>=2.7.3 in /usr/local/lib/python3.7/dist-packages (from pandas->pgmpy) (2.8.2)
Requirement already satisfied: pytz>=2017.3 in /usr/local/lib/python3.7/dist-packages (from pandas->pgmpy) (2022.1)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.7/dist-packages (from python-dateutil>=2.7.3->pandas->pgmpy) (
Requirement already satisfied: threadpoolctl>=2.0.0 in /usr/local/lib/python3.7/dist-packages (from scikit-learn->pgmpy) (3.1.0)
Requirement already satisfied: patsy>=0.4.0 in /usr/local/lib/python3.7/dist-packages (from statsmodels->pgmpy) (0.5.2)
Requirement already satisfied: typing-extensions in /usr/local/lib/python3.7/dist-packages (from torch->pgmpy) (4.1.1)
Installing collected packages: pgmpy
Successfully installed pgmpy-0.1.19
```

from pgmpy.models import BayesianNetwork from pgmpy.factors.discrete import TabularCPD import networkx as nx

```
1 from pgmpy.models import BayesianNetwork
2 from pgmpy.factors.discrete import TabularCPD
3 import networkx as nx
4 import pylab as plt
```

The BayesianModel can be initialized by passing a list of edges in the model structure. In this case, there are 4 edges in the model: Guest--> Host, Price-->host

```
1 # Defining Bayesian Structure
2 model = BayesianNetwork([('Guest', 'Host'), ('Price', 'Host')])
```

Step 2: Define the CPDs(Conditional Probability Distribution)

```
P(C):
+----------+----------+----------+----------+
|    C     |    0     |    1     |     2    |
+----------+----------+----------+----------+
|          |   0.33   |   0.33   |   0.33   |
+----------+----------+----------+----------+

P(P):
+----------+----------+----------+----------+
|    P     |    0     |    1     |     2    |
+----------+----------+----------+----------+
|          |   0.33   |   0.33   |   0.33   |
+----------+----------+----------+----------+

P(H | P, C):
+------+------+------+------+------+------+------+------+------+------+
|   C  |          0          |          1          |          2          |
+------+------+------+------+------+------+------+------+------+------+
|   P  |  0   |  1   |  2   |  0   |  1   |  2   |  0   |  1   |  2   |
+------+------+------+------+------+------+------+------+------+------+
| H=0  |  0   |  0   |  0   |  0   | 0.5  |  1   |  0   |  1   | 0.5  |
+------+------+------+------+------+------+------+------+------+------+
| H=1  | 0.5  |  0   |  1   |  0   |  0   |  0   |  1   |  0   | 0.5  |
+------+------+------+------+------+------+------+------+------+------+
| H=2  | 0.5  |  1   |  0   |  1   | 0.5  |  0   |  0   |  0   |  0   |
+------+------+------+------+------+------+------+------+------+------+
```

```
1 # Defining the CPDs:
2 cpd_guest = TabularCPD('Guest', 3, [[0.33], [0.33], [0.33]])
3 cpd_price = TabularCPD('Price', 3, [[0.33], [0.33], [0.33]])
4 cpd_host = TabularCPD('Host', 3, [[0, 0, 0, 0, 0.5, 1, 0, 1, 0.5],
5                           [0.5, 0, 1, 0, 0, 0, 1, 0, 0.5],
6                           [0.5, 1, 0, 1, 0.5, 0, 0, 0, 0]],
7                 evidence=['Guest', 'Price'], evidence_card=[3, 3])
```

Step 3: Add the CPDs to the model.

After defining the model parameters, we can now add them to the model using add_cpds method. The check_model method can also be used to verify if the CPDs are correctly defined for the model structure.

```
1 # Associating the CPDs with the network structure.
2 model.add_cpds(cpd_guest, cpd_price, cpd_host)
```

Now we will check the model structure and associated conditional probability distribution by the argument get_cpds() will return True if every this is fine else through an error msg.

```
1 model.check_model()
```

```
True
```

Now let's infer the network, if we want to check at the next step which door will the host open now. For that, we need access to the posterior probability from the network and while accessing we need to pass the evidence to the function. Evidence is needed to be given when we are evaluating posterior probability, here in our task evidence is nothing but which door is Guest selected and where is the Price.

```
1 # Infering the posterior probability
2 from pgmpy.inference import VariableElimination
3
4 infer = VariableElimination(model)
5 posterior_p = infer.query(['Host'], evidence={'Guest': 2, 'Price': 2})
6 print(posterior_p)
```

```
/usr/local/lib/python3.7/dist-packages/statsmodels/tools/_testing.py:19: FutureWarning: pandas.util.testing is deprecated. Use t
  import pandas.util.testing as tm
```

Finding Elimination Order: : 0/0 [00:12<?, ?it/s]

0/0 [00:00<?, ?it/s]

```
+---------+-------------+
| Host    |   phi(Host) |
+=========+=============+
| Host(0) |      0.5000 |
+---------+-------------+
| Host(1) |      0.5000 |
+---------+-------------+
| Host(2) |      0.0000 |
+---------+-------------+
```

```
1 # Infering the posterior probability
2 from pgmpy.inference import VariableElimination
3
4 infer = VariableElimination(model)
5 posterior_p = infer.query(['Host'], evidence={'Guest': 1, 'Price': 2})
6 print(posterior_p)
```

Finding Elimination Order: : 0/0 [00:00<?, ?it/s]

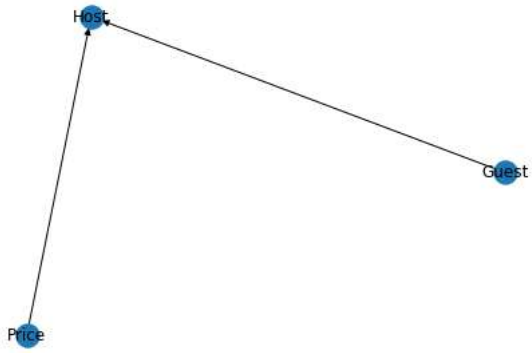
0/0 [00:00<?, ?it/s]

```
+---------+-------------+
| Host    |   phi(Host) |
+=========+=============+
| Host(0) |      1.0000 |
+---------+-------------+
| Host(1) |      0.0000 |
+---------+-------------+
| Host(2) |      0.0000 |
+---------+-------------+
```

The probability distribution of the Host is clearly satisfying the theme of the contest. In the reality also, in this situation host definitely not going to open the second door he will open either of the first two and that's what the above simulation tells.

Now, let's plot our above model. This can be done with the help of Network and Pylab. NetworkX is a Python-based software package for constructing, altering, and researching the structure, dynamics, and function of complex networks. PyLab is a procedural interface to the object-oriented charting toolkit Matplotlib, and it is used to examine large complex networks represented as graphs with nodes and edges.

```
1 nx.draw(model, with_labels=True)
2 #plt.savefig('model.png')
3 #plt.close()
```



[Reference1]

[Reference2]

Double-click (or enter) to edit